

Supplements for SciPy 2022

Rohit Goswami · 2022-11-15 · presentations · ramblings · fortran · python · posters

A meta-post on my in-person attendance of the 2022 SciPy Conference

Background

I hadn't been in an in-person conference for quite a long while now. There were multiple things going for SciPyCon. Collaborators at UT Austin (from the [EON project](#)). The prospect of meeting folks [from Quansight](#). Meeting other members of the [NumPy community](#) I'd been working with virtually for a year. I had the pleasure of being the recipient of a [SciPy Scholarship](#) for [my poster](#) and [conference proceeding](#) on [Wailord](#), which sealed the deal for me. I had the privilege of representing every facet of my interests, including a poster on LPython (an LFortran spin-off) and a talk on F2PY (with a distinguished [panel of maintainers](#)).

Maintaining Fortran in Python in Perpetuity

Co-Authors

[Dr. Melissa Mendonca](#) (Quansight Labs), [Dr. Ralf Gommers](#) (Quansight Labs), [Thirumalai Shaktivel](#) (Fortran-Lang / LFortran), [Dr. Pearu Peterson](#) (Quansight Labs)

Duration

20 minutes (15 + 5)

Medium

Maintainers Track Talk

Paper DOI [10.25080/majora-212e5952-021](#)

Slides DOI [10.5281/zenodo.7325023](#)

Abstract

Fortran forms the computational bedrock of the scientific community. Despite the prevalence of the pack libraries (LAPACK, PROPACK etc.) within SciPy and the fact that `f2py` has been an integral part of the NumPy ecosystem for decades now, it is seldom brought up. In particular, newer, younger programming languages and projects are consistently brought up as possible contenders to the crown of numerical computing within the Scientific Python ecosystem. Being such a foundational project has a unique set of maintainability challenges as well. Hyrum's law rings especially true for `f2py`, scientific projects, both public and private, depend heavily on every aspect of the exposed API. A boon for scientific programming from the very beginning, `f2py` has enabled high performance computing without in-depth understanding of either the Python-C API or the Fortran language itself. Even as `f2py` reinvents itself for modern Fortran constructs however, backwards compatibility requirements remain (e.g. `scipy` lags by two versions). From a brief historical overview of the scientific python ecosystem and interoperability issues we will move through the sustainability efforts through the years. We will cover changes in `f2py`, both planned and implemented, including the migration towards the Python-C Limited API, the newer `argparse` front-end, moving away from `np.distutils`, restructuring and reducing reliance on features

implemented by the NumPy-C API, and supporting newer Fortran standards (with derived types being the poster child of missing Modern Fortran features). The talk will also briefly cover the code-generation and fortranobject constructs. The focus is not meant to be the internals so much as a 50-foot view of each constituent part of `f2py` and its long term viability. Part of the discussion will revolve around synergies with existing projects, both within the scientific Python ecosystem (`scipy`), and Fortran-Lang (LFortran), and by planning to involve younger developers via GSoC. Lastly, we will discuss how the FOSS community stepped up to support `f2py` and how we intend to continue to carry modern Fortran as a corner stone of the scientific python ecosystem for many years to come in no small part by providing a common set of interoperability standards in the form of NEPs (NumPy Enhancement Proposals).

Short Summary

This talk covers recent advances to the `f2py` project, including preliminary derived type support, better documentation, and restructured design to improve new contributor experiences. Additionally, we will discuss the structure and role of interoperability libraries and how modern Fortran intends to harmoniously and symbiotically evolve within the Python ecosystem in the context of Fortran-Lang, the `np.distutils` deprecation, newer compute devices and the ever changing landscape of high performance programming.

Slides

- [Best viewed here](#) using a browser (in a new tab)
- A `pdf` copy of the slides are embedded below
- The `orgmode` source [is here on the site's GH repo](#)

Video

LPython: Interactive LLVM-based Python Compiler for Modern Architectures

Co-Authors

[Ondřej Čertík](#) (GSI Technology), [Brian Beckman](#) (GSI Technology), [Naman Gera](#) (IIT Guwahati), [Smit Lunagariya](#) (IIT BHU), [Gagandeep Singh](#) (GSI Technology), [Thirumalai Shaktivel](#) (Fortran-Lang / LFortran), [Dylon Edwards](#) (GSI Technology)

Medium

General Track Poster

DOI [10.5281/zenodo.7325061](https://doi.org/10.5281/zenodo.7325061)

Abstract

We demonstrate to the community that it is possible to use Python in a modern interactive way and yet have execution speed as fast or faster than other compiled languages such as C++ or Fortran. Python is one of the most used languages today. For performance applications such as High Performance Computing (HPC) or any other kind of numerical computing the standard CPython implementation is often not fast enough and it is difficult to run Python code on GPUs and other accelerators. To address these issues we have developed LPython, a Python compiler that can compile Python code to binaries, work interactively, and run on all platforms.

LPython is written in C++ and it has multiple backends to generate code including LLVM [1] and C++. The compiler has been open sourced under BSD license, available at <https://github.com/lcompilers/lpython>. The Abstract Syntax Tree (AST) and the intermediate Abstract

Semantic Representation (ASR) is represented using the ASDL domain-specific language [2], just like CPython's AST. LPython is designed as a library with separate building blocks (parser, AST, ASR, semantic phase, codegen) that are all exposed to the user/developer in a natural way to make it easy to contribute back. LPython is using the same internal representation (ASR) as in LFortran [3], and both the LPython and LFortran frontends are effectively surface languages that share the same middle end and backends, as well as high and low level optimizations. Both LPython and LFortran are part of LCompilers [4]. The speed of LPython comes from high level optimizations at the ASR level, as well as the low level optimizations that LLVM can do. In addition it is remarkably easy to customize back ends.

Short Summary

We are developing a modern open-source Python compiler called LPython (<https://lpython.org/>) that can execute user's code interactively in Jupyter to allow exploratory work (much like CPython, MATLAB or Julia) as well as compile to binaries with the goal to run user's code on modern architectures such as multi-core CPU, GPU, as well as unfamiliar, new architectures like GSI's APU, which features programmable compute-in memory. We aim to provide the best possible performance for numerical array oriented code. Live demo in a Jupyter notebook will be shown. The compiler itself is written in C++ for robustness and speed.

Wailord: Parsers and Reproducibility for Quantum Chemistry

Medium

Materials & Chemistry Poster

DOI [10.5281/zenodo.7325038](https://doi.org/10.5281/zenodo.7325038)

Abstract

The concept of a crisis of reproducibility in scientific research needs no introduction. Although there are several tooling approaches one can take to reduce the cognitive load of keeping track of various steps of an analysis pipeline [1], there remains an almost linguistic gap when it comes to interfacing with domain specific tools.

We demonstrate the role of parsers in the reproducibility workflow. By focusing on the generation of input files and the structured extraction of output data, we will aim to plug a gap in the generation of reproducible reports, namely, interfacing (via file I/O) with existing software. The file I/O interface justifiably has many detractors, especially on an HPC (high performance computing) cluster, I/O can be a bottleneck. However, when faced with an opaque binary which outputs freeform results, powered by an input file which has little to no structure beyond a 1500 page manual of keyword arguments, the utility of a domain specific parser can pay off immensely. In our quest to translate domain intuition into computational input constraints, we will work in a reduced grammar, an intermediate representation (IR). Such an IR can be generated for multiple program specifications, so extensions to other software is not difficult either.

As a concrete realization of an abstract concept, we will discuss Wailord [2], which uses parsimonious [3] and cookiecutter [4] to interface with ORCA [5], a popular free (but not open source) quantum chemistry software suite. Wailord. We will go over how such an input generation and output parser technique allows for catching otherwise hard to track down errors. Taking a step away from the problem of writing single-purpose input files and functionalities, we demonstrate how a series of tasks can be defined, executed, and harvested into a single report, at the cost of giving up control over the folder structure.

Short Summary

Much of the scientific python ecosystem deals with problems at the level when their structure is already present in memory. However, the generation of input files for driving existing codes, as well as the parsing of results is not typically covered in great detail. This submission bridges the gap between external programs and data-structures. This shows, via a practical example, the utility of code-generation and parsing expression grammar parsers for reproducible results in quantum chemistry.

Thoughts

SciPyCon exceeded my expectations. Everyone was kind, and I met a good number of people from QS and other places who hopefully did not come away with a poor impression of me. There was a last minute [comic rush](#) which [Mars](#) arranged. I also ended up [chatting with Jonathan Fine](#) to discuss NumPy [dropping PDF documentation builds](#). Plus I snagged a [Scientific Python tee-shirt](#). With [Ondřej Čertík](#) and [Antonio Cuni](#), thanks to a suggestion from [Travis](#) there were some neat discussions on LPython. Met with Sebastian and Inessa and Mars and Chuck among other NumPy folks. Reconnected with some people who've gone across the pond, like [PyIron's](#) Dr. Jan Janssen. Interacted with a bunch of people from the computational chemistry minisymposium in-spite of originally planning to stay away from (comp-chem) shop talk. This post is harder to write since after-all, SciPyCon was almost four months ago. Nevertheless, this is a supplements post. Presentations of projects ought to be tracked, for posterity and the projects themselves. It was good to have the opportunity to highlight projects I believe in, and to introduce [f2py's](#) GSoC student ([Namami Shankar](#)) to the community. I suppose, if invited, I would return.

Reading list

The talks, posters and software deposits behind the SciPy 2022 material, collected as a citation list: [4 references](#). The collection exports to BibTeX or any CSL style, so it can go straight into a bibliography.